

NULL Behavior in Joins

One of the most misunderstood SQL topics.

15.18.1 NULL = NULL ?

```
SELECT NULL = NULL;
```

Result

UNKNOWN

Not TRUE.

Not FALSE.

SQL uses three-valued logic.

Example

Customers

customer_id	name
1	Arpit
NULL	Aadz

Orders

customer_id
NULL

```
SELECT *
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id;
```

Output

No rows

Because

NULL = NULL



UNKNOWN

IS NULL vs = NULL

Wrong

```
WHERE managerId = NULL
```

Returns nothing.

Correct

```
WHERE managerId IS NULL
```

NULL-safe Comparison

MySQL

```
a <=> b
```

TRUE when both are NULL.

Best Practices

✓ Use

```
IS NULL
```

Never

= NULL

ON vs WHERE (Most Important Join Interview Topic)

Probably the single most asked SQL join interview question.

Students

id	name
1	Arpit
2	Aadz

Enrollment

student_id	course
1	DBMS

Query 1

```
SELECT *  
FROM Students s  
LEFT JOIN Enrollment e  
ON s.id=e.student_id;
```

Output

student	course
Arpit	DBMS
Aadz	NULL

Query 2

```
SELECT *  
FROM Students s  
LEFT JOIN Enrollment e  
ON s.id=e.student_id  
WHERE e.course='DBMS';
```

Output

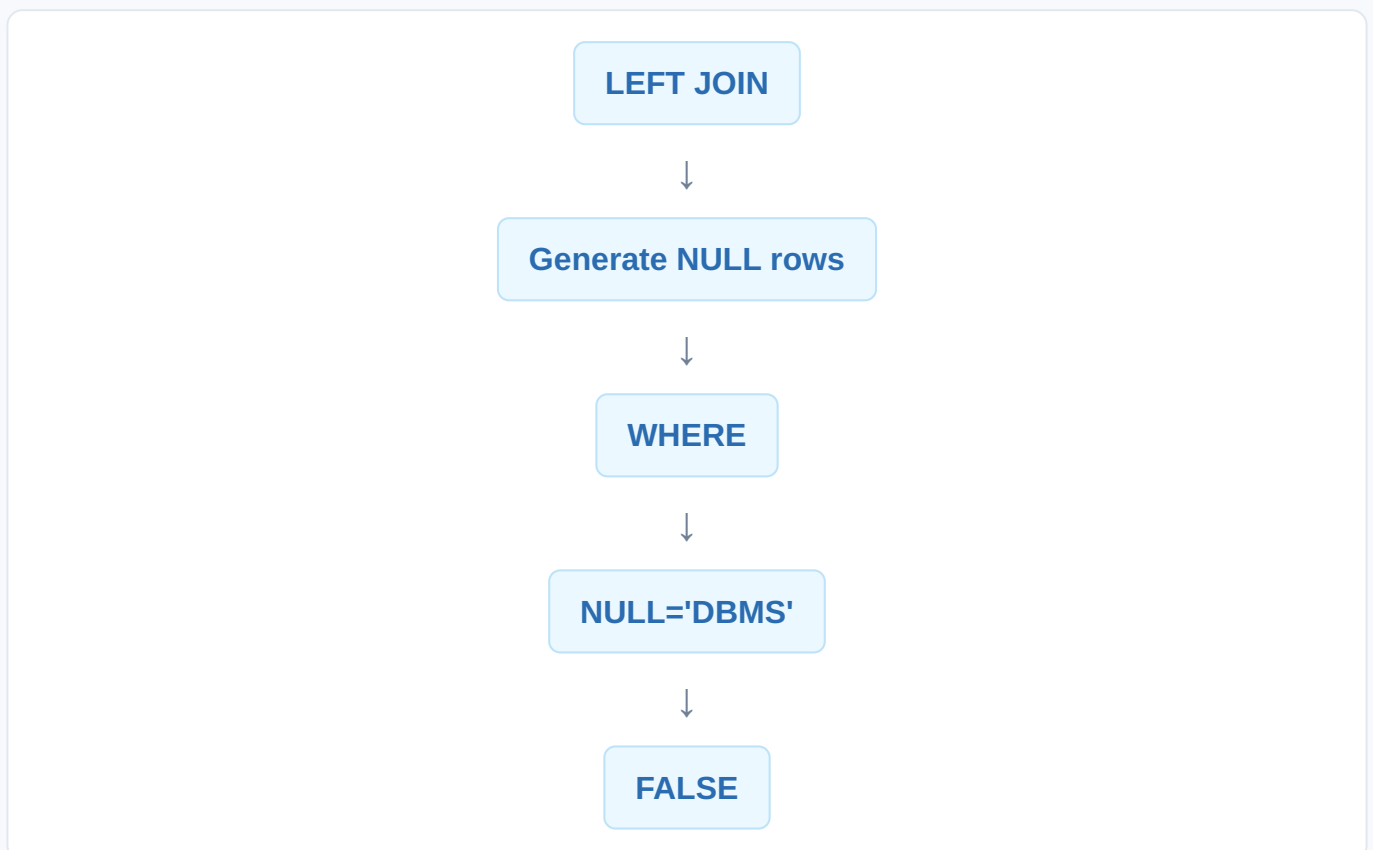
student	course
Arpit	DBMS

Aadz disappeared.

LEFT JOIN became INNER JOIN.

Why?

Execution



Correct

```
SELECT *
FROM Students s
LEFT JOIN Enrollment e
ON s.id=e.student_id
AND e.course='DBMS';
```

Output

student	course
Arpit	DBMS
Aadz	NULL

Huge interview favorite.

Join Execution Internals

SQL doesn't simply compare rows one by one.
Optimizer chooses an algorithm.

Nested Loop Join



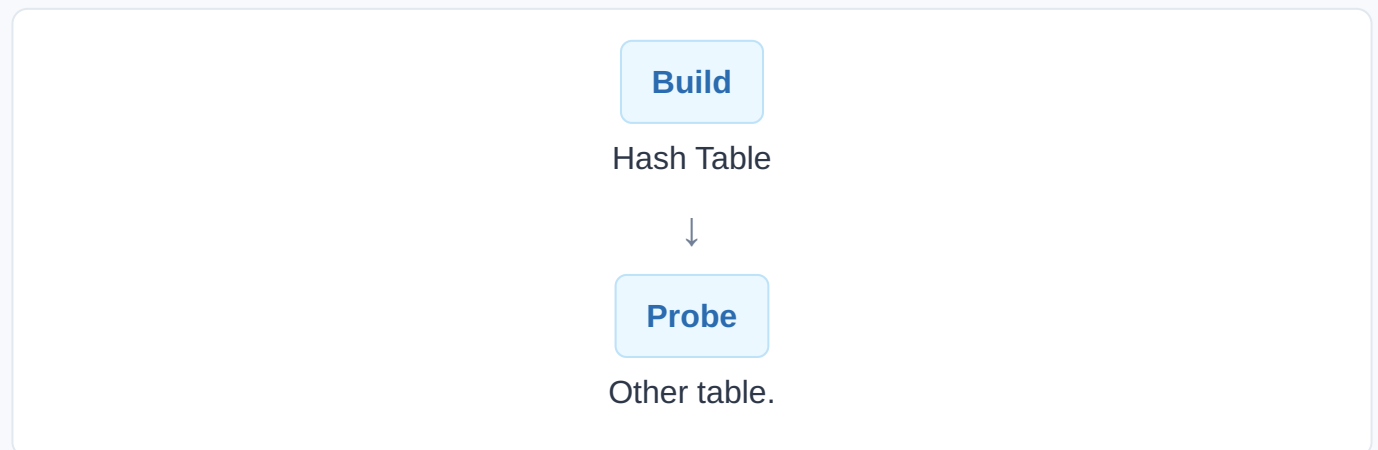
Complexity

$O(n \times m)$

Good when indexes exist.

Hash Join

Most common.



Average

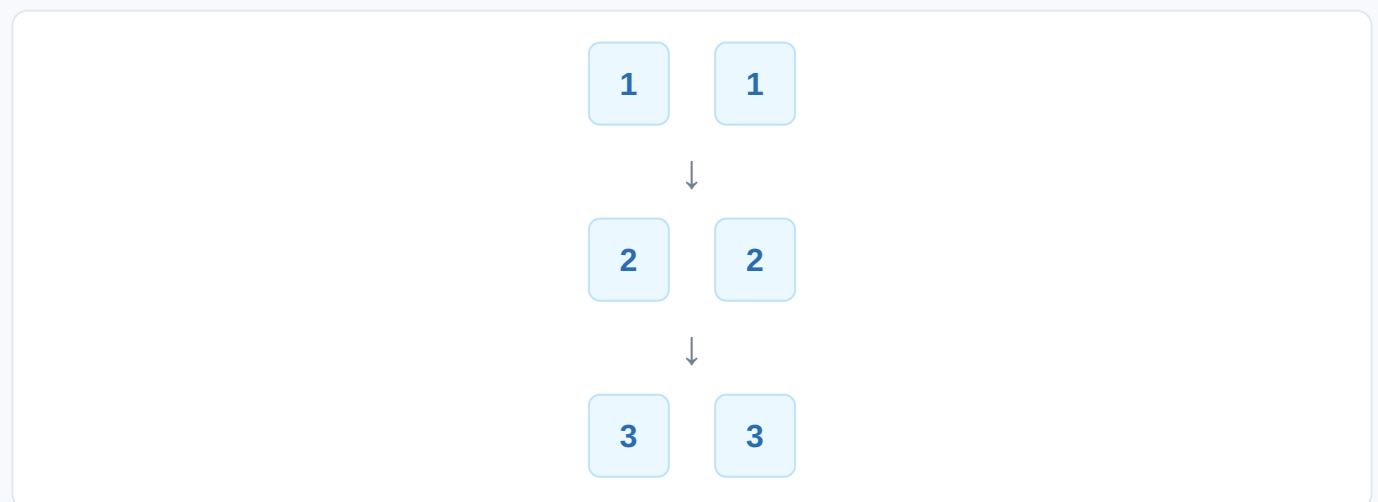
$O(n+m)$

Merge Join

Requires

Sorted tables.

Walk both together.



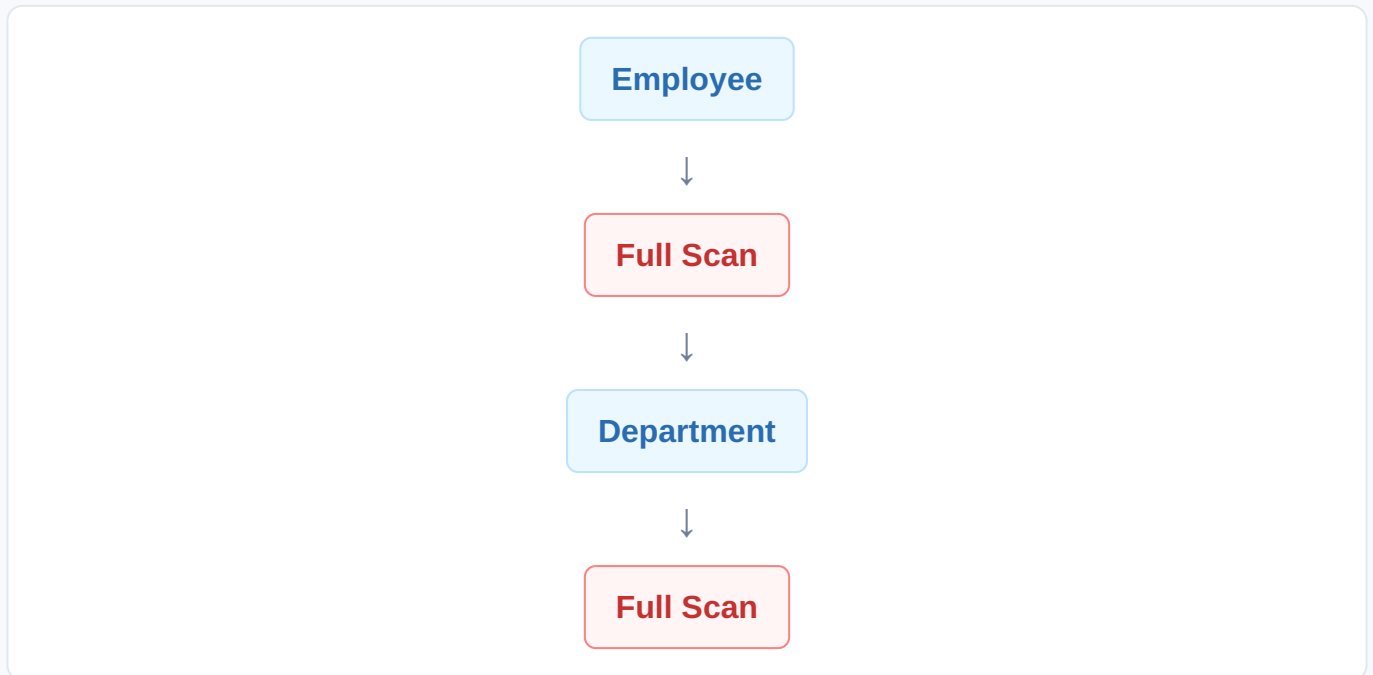
Very efficient.

Join Performance

Why joins become slow.

Missing Index

Without index



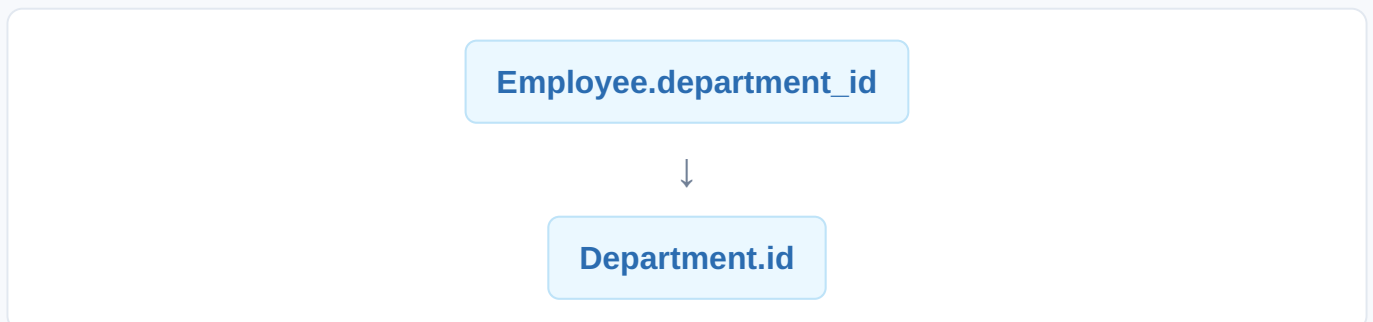
Very expensive.

Foreign Key Index

Always index

```
employee.department_id
```

Joining



becomes much faster.

Composite Index

Join

```
ON product_id  
AND year
```

Use

```
(product_id, year)
```

index.

Join Selectivity

High selectivity



Few rows match



Fast

Low selectivity



Millions match



Slow

Join Pitfalls & Edge Cases

Duplicate Rows

Customers

id
1

Orders

customer
1
1
1

INNER JOIN

Produces
Three rows.

Join Comparison Table

Join	Returns
INNER JOIN	Only matching rows from both tables
LEFT JOIN	All rows from the left table + matching rows from the right; unmatched right values become NULL
RIGHT JOIN	All rows from the right table + matching rows from the left; unmatched left values become NULL
FULL OUTER JOIN	All rows from both tables; unmatched sides are filled with NULL
CROSS JOIN	Cartesian product (every left row × every right row)
SELF JOIN	A table joined with itself using aliases
NATURAL JOIN	Automatically joins on all columns with the same names
SEMI JOIN	Returns only left-table rows that have at least one match (EXISTS / IN)
ANTI JOIN	Returns only left-table rows with no matching right-table rows (NOT EXISTS / LEFT JOIN ... IS NULL)